

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

INFORME TÉCNICO INDIVIDUAL
CÓDIGO DE ACTIVIDAD: TA-IND-04

Análisis de Rendimiento Paralelo aplicado al Proyecto Fin de Curso

Asignatura:	Aplicaciones Distribuidas (ISR-701) – Unidad 4
Docente Responsable:	Gleiston C. Guerrero-Ulloa, M.Sc.
Estudiante:	Juliana Emanuel
Equipo PE-U4 de Origen:	Equipo BCEL
PFC de Referencia:	BCEL – SGA Escuela Provincias Unidas
Componente del PFC:	Microservicio de Estudiantes / Matrículas
Transformación Declarada:	T1 – Filtrado y Selección de Datos Masivos
Repositorio Individual:	https://github.com/[usuario-compaera]/ta-ind-04-[apellido]
Commit SHA de Origen:	3711eda

1. Introducción y Contexto

El presente trabajo técnico individual analiza la aceleración y eficiencia computacional obtenida en la práctica GA-SUM-05 / PE-U4, enfocándose en la evaluación de técnicas de filtrado masivo sobre un pipeline distribuido en Apache Spark frente a la ejecución secuencial en pandas zaharia2016.

Los datos experimentales primarios provienen del repositorio oficial del **Equipo BCEL** (<https://github.com/BedonViteri/pe-u4-spark-BCEL>), registrados bajo el commit SHA 3711eda. Las pruebas se llevaron a cabo en Google Colab utilizando PySpark 3.5.0 configurado con 4 ejecutores sobre una carga de 10,6 millones de registros académicos.

El contexto de aplicación corresponde al Proyecto Fin de Curso **BCEL – SGA Escuela Provincias Unidas**, orientándose al módulo de consultas transaccionales de matrículas y estado académico.

2. Análisis Propio de los Resultados de Speedup

En estricto apego a la regla de individualidad, este informe aborda de forma exhaustiva la **Transformación T1: Filtrado y Selección de Datos Masivos**. En el sistema académico, T1 permite aislar rápidamente estudiantes activos, filtrar periodos lectivos específicos y depurar registros inconsistentes sobre el historial global.

2.1. Tabla de Datos Base (Plantilla Anexo A)

La Tabla 1 detalla los tiempos medianos de ejecución registrados en el commit 3711eda para las 5 transformaciones evaluadas.

Cuadro 1: Tabla de datos base de tiempos de ejecución y speedup ($N = 4$ ejecutores)

Id.	Transformación	T_{pandas} (s)	T_{spark} (s)	Speedup (S)
T1	Filtrado y selección (Foco)	12,45	2,10	5,93
T2	Agrupación y agregación	18,30	3,45	5,30
T3	Join de dos DataFrames	45,10	9,20	4,90
T4	Columna derivada compleja	15,80	2,80	5,64
T5	Ordenamiento y top-N	8,90	1,95	4,56
Escalado de la transformación foco (T1):				
	Foco con $N = 1$ ejecutor	12,45	5,80	2,15
	Foco con $N = 2$ ejecutores	12,45	3,60	3,46
	Foco con $N = 4$ ejecutores	12,45	2,10	5,93

2.2. Evaluación del Speedup frente a la Ley de Amdahl

La aceleración teórica de un sistema paralelo se modela mediante la Ley de Amdahl amdahl1967:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p} \quad (1)$$

Para T1, asumiendo una fracción paralelizable del algoritmo $p = 0,90$ y $N = 4$ ejecutores, el speedup teórico estimado por (1) resulta en:

$$S_{\text{teórico}}(4) = \frac{1}{(1-0,90) + \frac{0,90}{4}} = \frac{1}{0,10 + 0,225} \approx 3,08 \quad (2)$$

La desviación relativa Δ entre el speedup experimental ($S_{\text{med}} = 5,93$) y el teórico ($S_{\text{teo}} = 3,08$) es:

$$\Delta = \frac{S_{\text{med}} - S_{\text{teo}}}{S_{\text{teo}}} \times 100 \% = \frac{5,93 - 3,08}{3,08} \times 100 \% = +92,53 \% \quad (3)$$

Respuesta a la Pregunta Orientadora 1: La aceleración observada **supera significativamente la predicción teórica de Amdahl** (+92,53 %). Esto se debe a que la transformación T1 es un filtro embrutecidamente embarazoso (*narrow transformation*) que no requiere intercambio de datos entre nodos (*shuffle*). Spark evalúa los filtros de forma perezosa (*lazy evaluation*) y descarta particiones completas en memoria mediante empuje de predicados (*predicate pushdown*), mientras que pandas procesa el conjunto completo de datos en un solo hilo saturando la memoria caché de la CPU **ahmed2020**.

3. Diagnóstico de la Fracción No Escalable

Para cuantificar la sobrecarga del motor en operaciones de filtrado, se aplica la Métrica de Karp-Flatt **karp1990**:

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1 \quad (4)$$

Junto con la eficiencia del paralelismo $E(N)$:

$$E(N) = \frac{S(N)}{N} \quad (5)$$

Sustituyendo las mediciones de escalado de T1 para $N = 2$ ($S_2 = 3,46$) y $N = 4$ ($S_4 = 5,93$) en la ecuación (4):

$$e(2) = \frac{\frac{1}{3,46} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0,2890 - 0,5000}{0,5000} = -0,4220 \quad (6)$$

$$e(4) = \frac{\frac{1}{5,93} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{0,1686 - 0,2500}{0,7500} = -0,1085 \quad (7)$$

Respuesta a la Pregunta Orientadora 2: La métrica de Karp-Flatt registra una clara tendencia creciente (pasando de $-0,4220$ a $-0,1085$). Al tratarse de una transformación sin *shuffle*, la ligera pérdida de escalabilidad no es algorítmica, sino consecuencia directa de la **sobrecarga de coordinación del motor** (distribución de tareas desde el Driver, serialización de objetos y gestión de hilos por ejecutor) **karp1990**.

4. Propuesta de Integración al PFC

4.1. Caso de Uso y Arquitectura de Integración

En la arquitectura del SGA BCEL, la búsqueda de registros históricos de alumnos sobre tablas masivas degrada las consultas relacionales del sistema. Se propone integrar Spark como un servicio de filtrado asíncrono para la extracción de subconjuntos de datos antes de generar los reportes de matrículas.

5. Dimensionamiento y Viabilidad

Para determinar el umbral crítico n^* , se iguala el tiempo secuencial $T_{\text{sec}}(n) = \alpha n$ con el distribuido $T_{\text{dist}}(n) = \beta + \gamma \frac{n}{N}$ **karp1990**:



Figura 1: Aceleración experimental observada por transformación ($N = 4$ ejecutores) centrada en T1.

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N} \quad (8)$$

Ajustando los parámetros para T1 ($\alpha = 1,17 \times 10^{-6}$ s/reg, $\beta = 1,65$ s, $\gamma = 2,0 \times 10^{-7}$ s/reg y $N = 4$):

$$n^* = \frac{1,65}{1,17 \times 10^{-6} - \frac{2,0 \times 10^{-7}}{4}} = \frac{1,65}{1,17 \times 10^{-6} - 5,0 \times 10^{-8}} \approx 1,473,214 \text{ registros} \quad (9)$$

Respuesta a la Pregunta Orientadora 3 y 4: El filtrado distribuido con Spark en el módulo de registros resulta rentable a partir de ****1.47 millones de registros****. Para conjuntos de datos menores a 1,47 millones, las cláusulas ‘WHERE’ indexadas en la base de datos relacional (PostgreSQL) son más eficientes debido a la ausencia de la sobrecarga de inicialización ($\beta = 1,65$ s).

6. Postura Crítica y Alternativas Descartadas

Apache Spark demuestra máxima eficiencia en T1 por la ausencia de intercambios de red **zaharia2016**. Se descartaron dos alternativas:

1. **Pandas en un Servidor dedicado:** Satural la RAM y falla ante crecimientos imprevistos de datos.
2. **Polars (Rust):** Excelente para un solo nodo, pero no ofrece escalabilidad nativa en clústeres distribuidos.

7. Conclusiones

1. La Transformación T1 obtuvo la mayor aceleración del pipeline ($5,93\times$), impulsada por la naturaleza sin **shuffle** del filtrado masivo y la evaluación perezosa en memoria.
2. La Métrica de Karp-Flatt demostró que la no escalabilidad se debe exclusivamente al **overhead** fijo del motor Spark durante el despacho de tareas.
3. El punto de equilibrio económico y técnico para implementar filtrado distribuido en el PFC se fijó en 1,47 millones de registros.

Declaración de Uso de Inteligencia Artificial Generativa

Se utilizó la herramienta Gemini para soporte en la sintaxis LaTeX, formato de ecuaciones numeradas y estructuración del archivo BibTeX. Todos los análisis cuantitativos y propuestas de diseño fueron desarrollados por la autora.